

UDP/TCP Echo Application

Computer Networks project, A.A. 2025/26
Feri Alessandro, 7081870

1. Introduction

For this assignment I developed a small command-line application in Rust. The program can run as a client or as a server, and it supports both UDP and TCP. I tried to keep the application simple to use from the terminal, because the main goal was not to build a big software, but to test the network behaviour in a clear way.

In server mode the program opens a port selected at run time, waits for incoming data, and sends back an echo response. In client mode the program sends a message to a host and port selected at run time, then waits for the answer. The number of messages is also selected at run time with an option.

I also added some extra features that were useful during the testing part. The server can modify the echo payload, for example converting it to uppercase or reversing it. The client can also add a timestamp to measure the Round Trip Time (RTT). For TCP, I added an option to disable Nagle's algorithm, so I could see if there was some difference in the results.

2. Program Structure

The executable has two main commands:

```
server <protocol> <port>
client <protocol> <host> <port> <message>
```

The protocol can be `udp` or `tcp`. The client option `--count` decides how many messages are sent. The server option `--transform` decides if the reply is sent back normally or modified. The possible values are `none`, `upper`, and `reverse`.

The UDP part is quite direct. The server creates a `UdpSocket`, receives one datagram, eventually applies the optional transformation, and sends one datagram back to the same address. This is natural for UDP, because UDP already keeps the message boundaries: one datagram is one application message.

TCP was a little more tricky. At the beginning I treated each `TCP read()` like it was one complete message, but this is not correct because TCP is a stream. It can split or join data internally, so the application must decide where a message starts and ends. To solve this, I used a small framing format:

```
[4-byte big-endian payload length][payload bytes]
```

When a TCP message is received, the program first reads the 4-byte length field and then reads exactly that number of bytes. In this way the receiver can rebuild the original application message correctly. This also makes the TCP part interoperable with other implementations, if they use the same framing rule.

3. RTT Measurement

The optional `--rtt` flag enables the RTT measurement. When this option is used, the client puts an 8-byte timestamp at the beginning of the payload before sending the message. The server keeps these first 8 bytes unchanged and applies the transformation only to the real text message after the timestamp.

The timestamp used in the code is in NTP64 format: 32 bits for seconds and 32 bits for the fractional part. In practice, the program starts from the Unix time given by the operating system and converts it using the offset between 1900 and 1970.

For each valid answer, the client prints the RTT and the received message. At the end it also prints a small summary:

```
Avg RTT
Max RTT
Sent
Received
Unanswered
```

Unanswered is calculated as `Sent - Received`. For UDP this can mean that a packet was lost or that the timeout expired. For TCP it usually means that the application did not receive the expected reply, even if TCP itself manages retransmissions under the hood.

4. Test Setup

I tested the application both locally and with a remote server on Amazon EC2. For the measurements in this report, the server was running on an EC2 instance in the Frankfurt region, while the client was running from my computer. In the security group I opened UDP port 8888 and TCP port 9999. The public IP used for the tests was 52.29.154.138.

The main RTT tests were made with 1000 messages for each run and with `--rtt` enabled. For the basic UDP and TCP tests, I started two clients in parallel, so the server had to handle more than one client at the same time. This was useful also to check that the TCP server did not block on a single connection.

Example commands:

```
cargo run -- server udp 8888 --rtt
cargo run -- client udp 52.29.154.138 8888 ciao --count 1000 --rtt

cargo run -- server tcp 9999 --rtt
cargo run -- client tcp 52.29.154.138 9999 ciao --count 1000 --rtt
```

I also repeated the TCP tests with `--no-delay`, which enables `TCP_NODELAY`. This disables Nagle's algorithm. I wanted to check if this setting changed something for small echo messages, even if is not his domain.

5. Results

The first table shows the comparison between normal UDP and TCP runs.

Protocol	Run	Sent	Received	Unanswered	Avg RTT	Max RTT
UDP	1	1000	1000	0	20.012 ms	25.005 ms
UDP	2	1000	1000	0	19.327 ms	23.766 ms
TCP	1	1000	1000	0	20.227 ms	77.253 ms
TCP	2	1000	1000	0	19.234 ms	75.241 ms

The average RTT is very similar for UDP and TCP, around 19-20 ms. So, in this test, there is not a big difference in the average value. The bigger difference is in the maximum RTT. TCP had some higher spikes, while UDP stayed more stable in these two runs.

The second table compares TCP with and without `TCP_NODELAY`.

TCP mode	Run	Sent	Received	Unanswered	Avg RTT	Max RTT
Default	1	1000	1000	0	20.227 ms	77.253 ms
Default	2	1000	1000	0	19.234 ms	75.241 ms
No delay	1	1000	1000	0	20.091 ms	24.458 ms
No delay	2	1000	1000	0	19.233 ms	26.423 ms

With `TCP_NODELAY`, the average RTT is almost the same. The maximum RTT, instead, is much lower in these tests. I would not say that this is always true in every network, because it depends on the operating system and on the traffic situation, but in my case the effect was visible.

6. Echo Transform Checks

To check that the server was really processing the payload and not only sending back random data, I tested the two transformations implemented in the program. The message sent by the client was `ciao`.

Protocol	Transform	Received payload	Sent	Received	Unanswered
UDP	upper	CIA0	1000	1000	0
UDP	reverse	oaic	1000	1000	0
TCP	upper	CIA0	1000	1000	0
TCP	reverse	oaic	1000	1000	0

The results show that the transformations work for both protocols. They also show that the timestamp handling works correctly, because the server modifies only the message part and leaves the first timestamp bytes unchanged.

7. Interoperability

The assignment says that the application must be interoperable and encourages testing with other groups. I did not manage to complete a real test with another group, because I did not have another group available at the same time. Anyway, I kept the protocol simple so that this test could be done without too much confusion.

For UDP, interoperability is easy: the client sends one datagram and the server answers with one datagram. For TCP, the other implementation must use the same 4-byte big-endian length prefix before the payload. Without this rule, two programs can open the TCP connection correctly, but they may not agree on where each application message finishes.

8. Conclusion

At the end of the project, the application covers the main requirements of the assignment. It can work as both client and server, it supports UDP and TCP, the ports are chosen at run time, and the client can send a configurable number of messages. The echo behaviour works for both protocols, including the optional payload transformations.

The RTT tests were useful to compare the two protocols in a real network situation, not only on localhost. In my measurements, UDP and TCP had almost the same average RTT, around 19-20 ms. The main difference was in the maximum values: TCP had higher peaks in the default configuration, while the runs with `TCP_NODELAY` were more stable. This result is only valid for the test conditions I used, but it was still interesting to see the effect clearly.

The part that required the most attention was TCP. At first it is easy to think about TCP messages in the same way as UDP packets, but TCP is only a stream of bytes. Adding the 4-byte length prefix made the communication much more reliable, because the application can always know where one message ends and the next one starts.